fn make_float_to_bigint_threshold

Michael Shoemate (@Shoeboxam)

This proof resides in **"contrib"** because it has not completed the vetting process.

Proves soundness of the implementation of `make_float_to_bigint_threshold` in `mod.rs` at commit f5bb719 (outdated[1]).

1 Hoare Triple

Precondition

Compiler-verified

- Generic `TK` implements trait `Hashable`
- Generic `TV` implements trait `Float`
- Const-generic `P` is of type `usize`
- Generic `QI` implements trait `Number`
- Type `RBig` implements traits `TryFrom<TV>` and `TryFrom<QI>`. This is for fallible exact casting to rationals from floats in the function and input sensitivity in the privacy map.
- Type `i32` implements trait `ExactIntCast«T as FloatBits>::Bits>`, This requirement means that the raw bits of `T` can be exactly cast to an `i32`.

Caller-verified

None

Pseudocode

```
def make_float_to_bigint_threshold(
    input_space: tuple[
        MapDomain[AtomDomain[TK], MapDomain[TV]], L0PInfDistance[P, AbsoluteDistance[QI]]
    ],
    threshold: TV,
    k: i32,
) -> Transformation[
    MapDomain[AtomDomain[TK], AtomDomain[TV]],
    MapDomain[AtomDomain[TK], AtomDomain[IBig]],
    L0PInfDistance[P, AbsoluteDistance[QI]],
    L0PInfDistance[P, AbsoluteDistance[RBig]],
]:
    input_domain, input_metric = input_space
```

[1] See new changes with `git diff f5bb719..c6ec4181 rust/src/measurements/noise_threshold/nature/float/mod.rs`

```
    if input_domain.value_domain.nan():
        raise "input_domain hashmap values may not contain NaN elements"

    r_threshold = RBig.try_from(threshold)

    min_k = get_min_k(TV)
    if k < min_k:  #
        raise f"k ({k}) must not be smaller than {min_k}"

    def value_function(val):  #
        try:  #
            val = RBig.try_from(val)
        except Exception:
            val = RBig.ZERO

        return find_nearest_multiple_of_2k(val, k)  #

    def stability_map(d_in):
        l0, lp, li = d_in

        r_lp = RBig.try_from(lp)
        r_lp_round = get_rounding_distance(k, usize.from_(l0), P)
        r_lp = x_mul_2k(r_lp + r_lp_round, -k)  #

        r_li = RBig.try_from(li)
        if r_li > x_mul_2k(r_threshold, -k): #
            raise f"li ({li}) must not be larger than threshold ({threshold})"
        r_li_round = get_rounding_distance(k, 1, P)
        r_li = x_mul_2k(r_li + r_li_round, -k)  #

        return l0, r_lp, r_li

    return Transformation.new(
        input_domain,
        MapDomain(  #
            key_domain=input_domain.key_domain,
            value_domain=AtomDomain.default(IBig),
        ),
        Function.new(lambda x: {k: value_function(v) for k, v in x.items()}),
        input_metric,
        LOPI.default(),
        StabilityMap.new_fallible(stability_map),
    )
```

Postcondition

Theorem 1.1. For every setting of the input parameters (`input_space, k, TK, TV, P, QI`) to `make_float_to_bigint_thr` such that the given preconditions hold, `make_float_to_bigint_threshold` raises an error (at compile time or run time) or returns a valid transformation. A valid transformation has the following properties:

1. (Data-independent runtime errors). For every pair of members x and x' in `input_domain`, `invoke`(x) and `invoke`(x') either both return the same error or neither return an error.

2. (Appropriate output domain). For every member x in `input_domain`, `function`(x) is in `output_domain` or raises a data-independent runtime error.

3. (Stability guarantee). For every pair of members x and x' in `input_domain` and for every pair (`d_in`, `d_out`), where `d_in` has the associated type for `input_metric` and `d_out` has the associated type for
`output_metric`, if x, x' are `d_in`-close under `input_metric`, `stability_map`(`d_in`) does not raise an error, and `stability_map`(`d_in`) = `d_out`, then `function`(x), `function`(x') are `d_out`-close under `output_metric`.

Proof. In the definition of the function on line 23, `RBig.try_from` is infallible when the input is non-nan. The precondition for `find_nearest_multiple_of_2k` is satisfied by line 20, so `find_nearest_multiple_of_2k` is infallible. There are no other sources of error in the function, so the function cannot raise data-dependent errors.

The function also always returns a hashmap with the same keys, and IBig values, meaning the output of the function is always a member of the output domain, as defined on line 48.

The stability argument breaks down into three parts:

- The casting from float to rational on line 24 is 1-stable, because the real values of the numbers remain un-changed, meaning the distance between adjacent inputs always remains the same.

- The rounding on line 29 can cause an increase in the sensitivity equal to $\Delta_0^{1/p}(2^k - 2^{k_{min}})$.

$$\max_{x \sim x'} d_{Lp}(f(x), f(x')) \tag{1}$$
$$= \max_{x \sim x'} |\text{round}_k(x) - \text{round}_k(x')|_p \tag{2}$$
$$\leq \max_{x \sim x'} |(x + 2^{k-1}) - (x' - 2^{k-1} + 2^{k_{min}})|_p \tag{3}$$
$$\leq \max_{x \sim x'} |x - x'|_p + |R \cdot (2^k - 2^{k_{min}})|_p \qquad \text{where } R \in \{0,1\}^n \text{ with weight } \Delta_0 \tag{4}$$
$$= \max_{x \sim x'} d_{Lp}(x, x') + \Delta_0^{1/p}(2^k - 2^{k_{min}}) \tag{5}$$
$$= \texttt{d_in} + \Delta_0^{1/p}(2^k - 2^{k_{min}}) \tag{6}$$

  This increase in the sensitivity is reflected on line 36, which, by the postcondition of `get_rounding_distance`, returns the maximum increase in sensitivity due to rounding. The rounding distance is added to the L_p sensitivity.

  A similar analysis follows for L_∞ sensitivity on line 42, where only one rounding occurs instead of Δ_0. Notice that the check on line 39 is not necessary for the privacy guarantee, it improves the quality of the error message. The equivalent error raised from the core mechanism privacy map is not as user-friendly, because the constants are scaled by 2^k.

- The discarding of the denominator on line 29 is 2^k-stable, as the denominator is 2^k. This increase in sensitivity is also reflected on lines 36 and 42, where the sensitivity is multiplied by 2^{-k}.

For every pair of elements x, x' in `input_domain` and for every pair (`d_in`, `d_out`), where `d_in` has the associated type for `input_metric` and `d_out` has the associated type for `output_metric`, if x, x' are `d_in`-close under `input_metric`, `stability_map`(`d_in`) does not raise an exception, and `stability_map`(`d_in`) $\leq$ `d_out`, then `function`(x), `function`(x') are `d_out`-close under `output_metric`. □